

Programming a SAT solver (in C)

↳ should take in a boolean formula in CONJUNCTIVE Normal Form ^(CNF) _[satisfiable] and output whether it is solvable, or not.

for instance, a formula F in CNF form: Formula:
Clause: $\vee$ two
Literal

$$F = (\neg x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_3 \vee x_4)$$

or:

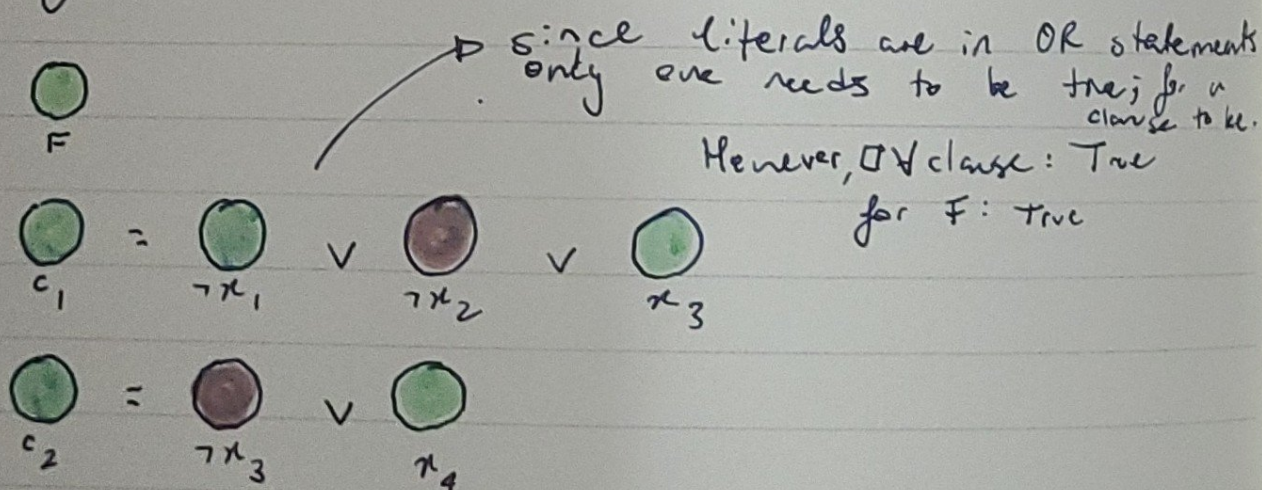
$$F = (-x_1 \vee -x_2 \vee x_3) \wedge (\neg x_3 \vee x_4)$$

a '-' is negation

to satisfy F , all clauses $\square T$ (and thus at least one literal per clause $\square T$).

~~True~~ **False**

the above formula:



so thus, a program [SAT solving] would output that this formula is satisfiable.

for a formula F with n vars, the interval for solutions

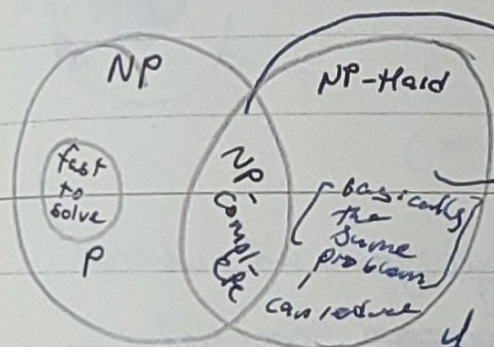
is $0 \leq n \leq 2^n$
 (unsatisfiable conditions) → theoretical maximum

k -literal clause:

eliminates $\frac{1}{2^k}$ assignments

As solving the BOOLEAN SATISFIABILITY PROBLEM (SAT) is quite difficult simply by toggling literals, there are several algorithms that optimize the process of result-finding.

It can take quite a long time, as it is NP-COMplete.



fast to check, hardest in NP.

all problems reduce to the same.

* NOTE:

as this is LITERALLY the million \$ Q we will come back to it later

if $P = NP$, every computational problem whose solution could be verified could also be solved quickly (in Polynomial Time) by a computer.

Crypto relies mainly on algs that are hard to solve, easy to check.

£1,000,000 price for the proof!

(come back to $P \stackrel{?}{=} NP$ later)

As mentioned, many optimisations exist for the SAT-solving algorithm. One is named the "BOOLEAN CONSTRAINT PROPAGATION" or BCP.

ppp →

Boolean Constraint Propagation (BCP)

↳ ... is ^{now} searching for a satisfying arrangement can be optimised by using "unit clauses".

↓
 $\forall L$ [except ONE] are set to 0 [False] leaving one unassigned literal whose value is therefore apparent (FORCED).

eg, $(x_1 \vee \neg x_2 \vee x_3)$ where SUBS $\left\{ \begin{array}{l} x_1 = 0 \\ x_2 = 1 \text{ (so } \neg x_2 = 0) \\ x_3 \text{ MUST} = 1 \end{array} \right.$
 $(\text{FALSE} \vee \text{FALSE} \vee x_3)$
↓
MUST be true (for the clause to be true)

So $\therefore$ a clause is unit under a partial assignment where the assignment makes all literals ^{on SAT} ~~undecided~~, except one which is undecided.

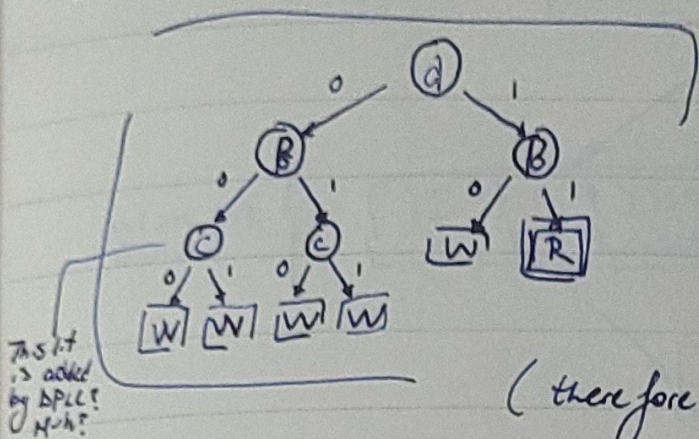
(like DPLL)
Algorithms repeat BCP until there $\rightarrow \exists \text{ unit}$

↳ One algorithm that uses this is DPLL, which I'll initially build the solver upon.

The Davis - Putnam - Logemann - Loveland (DPLL) algorithm

↳ ... is a SAT solver based on recursive backtracking that makes use of BCP... by mak[ing] BCP-driven inferences to cut down on the size of the search tree. When it encounters conflict, the last $\rightarrow$ BCP tec is switched to its negation
- (Unit clause to force $x_1 = 1$ or $x_2 = 0$)

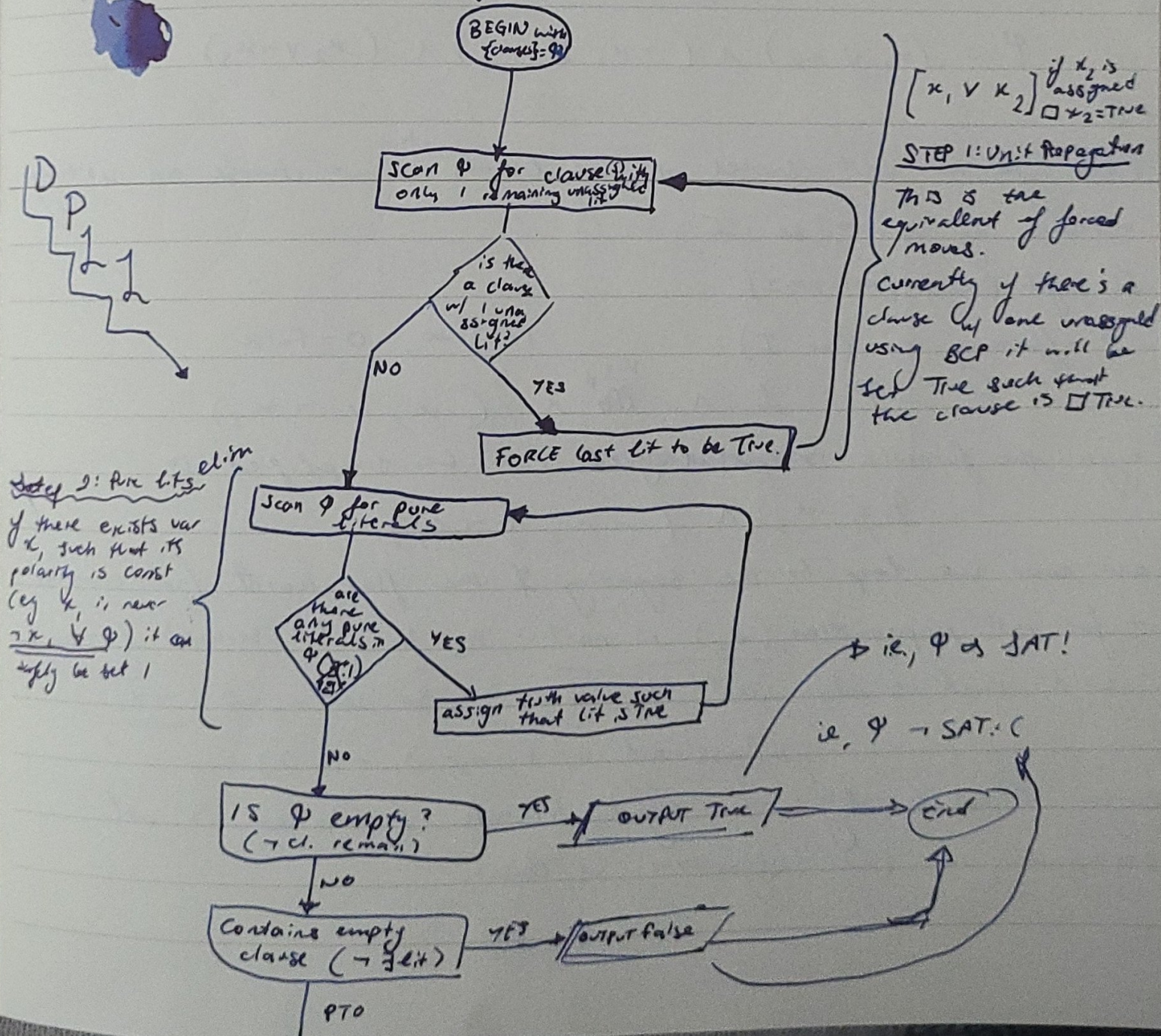
[diagram adapted from Wikipedia:]

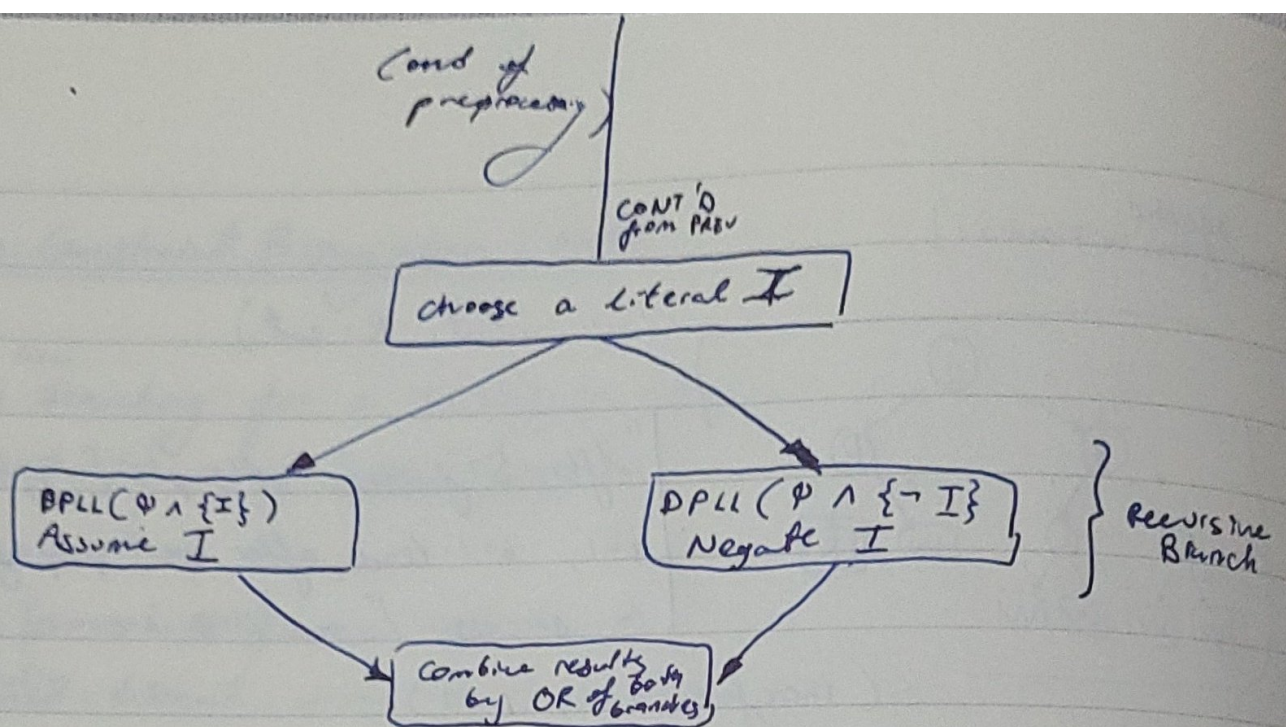


(W: $\rightarrow$ sat, R: sat)

"After 5 fruitless attempts [W], choosing $a=1, b=1$ leads after unit propagation to success. [$\rightarrow a + b + c$ is SAT].
- Wiki

if we let Φ be a set of clauses, a flowchart could be:





An example may be the formula:

$$\Phi = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (x_2 \vee \neg x_3)$$

①

There are no unit clauses, or pure literals, ^{etc.} so we choose an arbitrary variable, eg x_1 , to be I .

② Left Branch [AFFIRM I]

↳ affirming x_1 (or I): $1 = \text{True}, 0 = \text{False}$

$$\Phi = 1 \wedge \overset{x_3}{\cancel{(\neg x_1)}} \wedge (x_2 \vee \neg x_3)$$

ergo the formula ~~is not empty~~ can be simplified to

$$\Phi_{\text{left}} = x_3 \wedge (\neg x_2 \vee \neg x_3)$$

and now we loop to the beginning of the flowchart (unit clauses)

- for unit propagation, (x_3) is unit $\therefore x_3 = 1 \therefore$ C3 becomes $(\neg x_2)$

- and that is also unit! $(\neg x_2)$ so set $x_2 = 0 \therefore \neg x_2 = 1$

Φ (assignment $x_1 = 1, x_2 = 0, x_3 = 1$)

ergo Φ is empty! The right branch (Φ_{right}) is not engaged as the algorithm is done.

Another example: let $\varphi = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_1 \vee \neg x_3)$
 (no UCL, no pure lits, $\varphi \rightarrow \text{empty}$, no empty clauses), so Literal Choosing

① x_1 chosen to be var I.

② left branch forces $I = \text{TRUE}$

↳ ③ formula simplifies to

$$\varphi_{\text{left}} = \underline{(x_3) \wedge (\neg x_3)}$$

↳ ④ LOOP [GOTO start]: BCP
 as x_3 is unit, let $x_3 = 1$
 $\therefore \varphi$ is UNSAT as $\neg x_3$
 $\hookrightarrow \neg \text{SAT}$

↳ $\varphi_{\text{left}} = \text{UNSAT}$

↳ ⑤ Backtracking to φ_{right} [negate x_1]

↳ ⑥ ergo formula simplifies to

$$\varphi_{\text{right}} = [\text{just}] (x_2)$$

↳ ⑦ GOTO BCP.

(x_2) unit $\therefore x_2 = 1$ (affirmed)
 disappears (is TRUE for φ)

φ is empty

$\therefore \varphi_{\text{right}} = 1$

(right branch is true:

$x_1 = \text{FALSE}$
 $x_2 = \text{TRUE}$
 $x_3 = \text{FALSE}$
 (True works too))

PTO

C1: SAT by x_1
 C2: $\neg x_1 = 0$ leaves x_3
 C3: $\neg x_1 = 1$ leaves $\neg x_3$
 $(x_3) \wedge (\neg x_3) = \varphi_{\text{left}}$

C1: $x_1 = 0$ so leaving (x_2)
 C2: $x_1 = 1$ ✓
 C3: $x_1 = 1$ ✓ SAT

Implementing the DPLL Algorithm in C

Syntactically, there are a few things worth deciding.

① Formatting CNF formulas

All CNF formulas are composed of literals, clauses and the formula.
↳ (based upon assignment)

eg
$$\Phi = (\underbrace{x_1 \vee x_2}_{\text{Literal}}) \wedge (\underbrace{\neg x_1 \vee x_3}_{\text{Clause}}) \wedge (\underbrace{\neg x_1 \vee \neg x_3}_{\text{formula}})$$

Using the DIMACS CNF format seems best. The above would be:

formula.cnf

01	c	comment text		
02	p	cnf	3	3
03	1	2	0	
04	-1	3	0	
05	-1	-3	0	

Lines beginning with 'c' are comments.

Specifies problem of CNF format with 3 literals variables and 3 clauses
[IN THAT ORDER!]

(There would have been 6 literals)

Means end of clause.

variables denoted as

Signed ints where negative ints

are negations, i.e. x_1 is 1
 $\neg x_1$ is -1

For the programming, I feel like writing lines in the book is inefficient. The book will be used to plan from now.

The implementation may be ordered as such: - [after a lot of rubbing out my white board for the order]

① Core Maths Logic

- Scoreboard (hard assign an array vars such $[i]$ maps to 0, 1, or 2 (false, true, UNASSIGNED error values respectively)
- Evaluate literals (to check them)
- Evaluate clauses (ent. formula)

② Implement SAT

- UNIT Prep (forced ones)
- Base cases (w/ $\rightarrow$ backtrack)
- Branching (L vs R)

③ Reading, Writing

- Parse file in CNF
- Output SAT or $\rightarrow$, give VARS.

example clause: $x_1 \vee \neg x_2 \vee x_3$ CNF(R)
int clause[] = {1, -2, 3}

example assignment: $\begin{matrix} 2 & 1 & 0 \\ \text{UNASSIGNED} & & \text{FALSE} \\ \text{TRUE} & & \end{matrix}$ (F)

store assignment as:

int assignments[] = ~~{2, 1, 0}~~;

which shows $x_1 = 2$

$x_2 = 1$

$x_3 = 0$

so earlier
end of earlier

base at 1

{2, 2, 1, 0}

not used

* Implemented clause SAT
checker
clauses.c